

Izabela Rębisz

Temat Projektu:

Discord App – Webowy Komunikator Internetowy

Przedmiot: Aplikacje internetowe w Django

Spis treści

1. Opis projektu	3
1.1 Główne funkcjonalności systemu	3
1.1.1 Zarządzanie użytkownikami	3
1.1.2 Kanały komunikacyjne	3
1.1.3 Wiadomości prywatne	3
1.1.4 Reakcje i interakcje	3
1.1.5. Moderacja i administracja	3
1.2 Wykorzystane technologie	4
2. Instalacja i uruchomienie	4
3. Struktura plików	5
3.1 Najważniejsze pliki	5
3.2 Struktura aplikacji	5
6. Kod źródłowy	6
7. Przykłady działania programu	9
8. Podsumowanie	11

1. Opis projektu

Projekt „Discord App – Webowy Komunikator Internetowy” to aplikacja internetowa opracowana z wykorzystaniem frameworka Django oraz języka Python. System pełni funkcję prostego komunikatora inspirowanego platformą Discord, umożliwiając użytkownikom prowadzenie rozmów na kanałach publicznych oraz wymianę wiadomości prywatnych. Aplikacja została zbudowana w oparciu o architekturę MVT (Model–View–Template), charakterystyczną dla frameworka Django. W systemie zaimplementowano mechanizmy logowania i rejestracji użytkowników, obsługę kanałów komunikacyjnych, reakcje na wiadomości, profile użytkowników oraz podstawowe funkcje moderacyjne. W środowisku lokalnym projekt korzysta z bazy danych SQLite, natomiast w przypadku wdrożenia produkcyjnego możliwa jest integracja z bazą PostgreSQL.

1.1 Główne funkcjonalności systemu

1.1.1 Zarządzanie użytkownikami

System umożliwia rejestrację nowych użytkowników oraz obsługę procesu logowania i wylogowywania. Każdy użytkownik posiada własny profil, który może edytować po zalogowaniu do aplikacji. W projekcie zaimplementowano również mechanizm ról użytkowników, pozwalający na rozróżnienie zwykłych użytkowników, moderatorów oraz administratorów. Dodatkowo system wspiera możliwość blokowania użytkowników oraz monitorowania ich aktualnego statusu aktywności.

1.1.2 Kanały komunikacyjne

Aplikacja pozwala użytkownikom tworzyć kanały tekstowe służące do komunikacji grupowej. Użytkownicy mogą dołączać do wybranych kanałów i prowadzić rozmowy w czasie rzeczywistym. System udostępnia także funkcję wyszukiwania kanałów, co ułatwia odnajdywanie interesujących grup tematycznych.

1.1.3 Wiadomości prywatne

Projekt obsługuje komunikację prywatną pomiędzy użytkownikami w formie wiadomości typu Direct Message. System przechowuje historię prywatnych rozmów oraz umożliwia śledzenie liczby nieprzeczytanych wiadomości.

1.1.4 Reakcje i interakcje

W aplikacji zaimplementowano możliwość reagowania na wiadomości za pomocą emoji. Reakcje mogą być dodawane zarówno w kanałach publicznych, jak i w rozmowach prywatnych, co zwiększa interaktywność komunikacji pomiędzy użytkownikami.

1.1.5. Moderacja i administracja

System posiada funkcje administracyjne umożliwiające zarządzanie użytkownikami oraz kontrolę dostępu do wybranych funkcjonalności aplikacji. Administratorzy i moderatorzy

posiadają rozszerzone uprawnienia związane z moderowaniem treści oraz nadzorowaniem działania systemu.

1.2 Wykorzystane technologie

Projekt został wykonany z wykorzystaniem języka Python 3 oraz frameworka Django w wersji 5.2, który odpowiada za logikę backendową aplikacji oraz obsługę komunikacji z bazą danych. W środowisku lokalnym zastosowano bazę danych SQLite, wykorzystywaną do przechowywania informacji o użytkownikach, wiadomościach oraz kanałach komunikacyjnych.

Warstwa interfejsu użytkownika została przygotowana przy użyciu technologii HTML5 oraz CSS3, odpowiadających za strukturę oraz wygląd aplikacji internetowej. Do wdrożenia projektu wykorzystano platformę Render, umożliwiającą publikację aplikacji w środowisku produkcyjnym.

2. Instalacja i uruchomienie

W celu uruchomienia projektu należy pobrać i rozpakować archiwum ZIP zawierające aplikację. Następnie należy utworzyć środowisko wirtualne przy użyciu polecenia `python -m venv venv` oraz je aktywować.

Kolejnym krokiem jest instalacja wymaganych zależności z pliku `requirements.txt` przy użyciu polecenia `pip install -r requirements.txt`. Po poprawnym zainstalowaniu bibliotek należy wykonać migracje bazy danych komendą `python manage.py migrate`, a następnie uruchomić serwer developerski poleceniem `python manage.py runserver`.

Po uruchomieniu aplikacja będzie dostępna w przeglądarce pod adresem

<http://127.0.0.1:8000>.

Projekt jest również opublikowany na platformie Render i dostępny pod adresem

<https://discord-clone-sodo.onrender.com/login/>.

Przykładowe dane logowania:

Administrator:

Login: Izabela

Hasło: Admin12345

Moderator:

Login: Pawel

Hasło: Moderator12345

Użytkownik:

Login: Karol

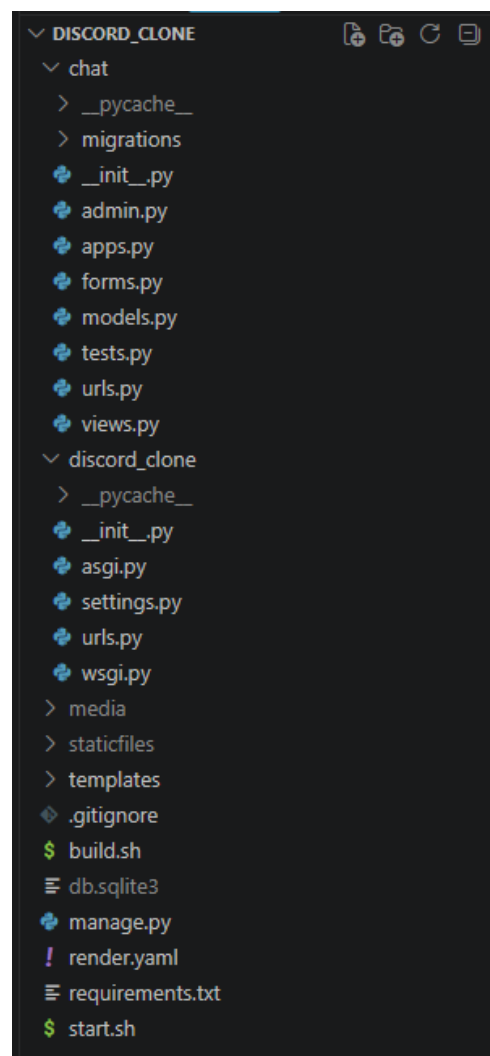
Hasło: User12345

3. Struktura plików

3.1 Najważniejsze pliki

Plik	Funkcja
manage.py	zarządzanie projektem Django
settings.py	konfiguracja aplikacji
urls.py	routing aplikacji
models.py	modele danych
views.py	logika biznesowa
forms.py	formularze Django
middleware.py	middleware aplikacji
requirements.txt	zależności projektu
render.yaml	konfiguracja wdrożenia

3.2 Struktura aplikacji



6. Kod źródłowy

Poniżej przedstawiono kod źródłowy wybranych plików aplikacji za pomocą zrzutów ekranu z Visual Studio Code.

models.py

```
4 class User(AbstractUser):
5     ROLE_CHOICES = [
6         ('admin', 'Admin'),
7         ('mod', 'Moderator'),
8         ('user', 'User'),
9     ]
10    role = models.CharField(max_length=10, choices=ROLE_CHOICES, default='user')
11    email = models.EmailField(unique=True)
12    avatar_url = models.URLField(blank=True)
13    bio = models.TextField(blank=True)
14    is_blocked = models.BooleanField(default=False)
15    last_seen = models.DateTimeField(blank=True, null=True)
16
17    class Channel(models.Model):
18        name = models.CharField(max_length=100)
19        members = models.ManyToManyField(User, blank=True, related_name='channels')
20
21        def __str__(self):
22            return self.name
23
24    class Message(models.Model):
25        user = models.ForeignKey(User, on_delete=models.CASCADE)
26        channel = models.ForeignKey(Channel, on_delete=models.CASCADE)
27        content = models.TextField(blank=True)
28        image_url = models.URLField(blank=True)
29        audio_url = models.URLField(blank=True)
30        created_at = models.DateTimeField(auto_now_add=True)
31
32
33    class Reaction(models.Model):
34        message = models.ForeignKey(Message, on_delete=models.CASCADE, related_name='reactions')
35        user = models.ForeignKey(User, on_delete=models.CASCADE)
36        emoji = models.CharField(max_length=20)
37        created_at = models.DateTimeField(auto_now_add=True)
38
39        class Meta:
40            unique_together = ('message', 'user', 'emoji')
41
42
43    class DirectMessage(models.Model):
44        sender = models.ForeignKey(User, on_delete=models.CASCADE, related_name='sent_direct_messages')
45        receiver = models.ForeignKey(User, on_delete=models.CASCADE, related_name='received_direct_messages')
46        content = models.TextField(blank=True)
47        image_url = models.URLField(blank=True)
48        audio_url = models.URLField(blank=True)
49        created_at = models.DateTimeField(auto_now_add=True)
50
```

urls.py

```
1  from django.urls import path
2  from . import views
3
4  urlpatterns = [
5      path('', views.home),
6      path('channel/<int:id>/', views.channel),
7      path('channel/<int:id>/join/', views.join_channel),
8      path('channels/new/', views.create_channel),
9      path('delete/<int:id>/', views.delete_message),
10     path('react/<int:id>/<str:emoji>/', views.react_message),
11     path('profile/', views.edit_profile),
12     path('block/<int:id>/', views.block_user),
13     path('unblock/<int:id>/', views.unblock_user),
14     path('dm/<int:user_id>/', views.direct_messages),
15     path('logout/', views.logout_view),
16     path('register/', views.register_view),
17 ]
```

settings.py

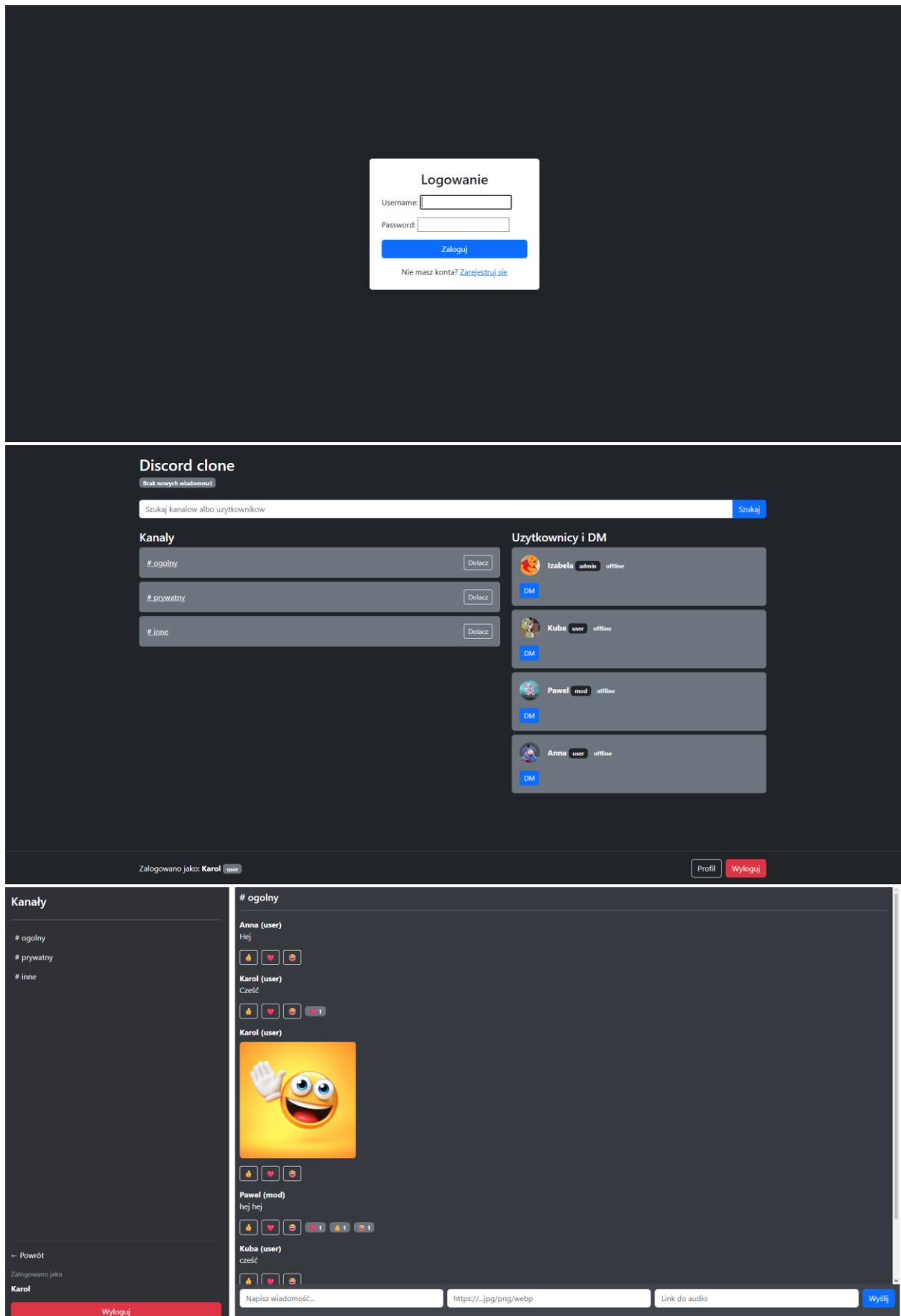
```
45  INSTALLED_APPS = [
46      'django.contrib.admin',
47      'django.contrib.auth',
48      'django.contrib.contenttypes',
49      'django.contrib.sessions',
50      'django.contrib.messages',
51      'django.contrib.staticfiles',
52      'chat',
53  ]
```

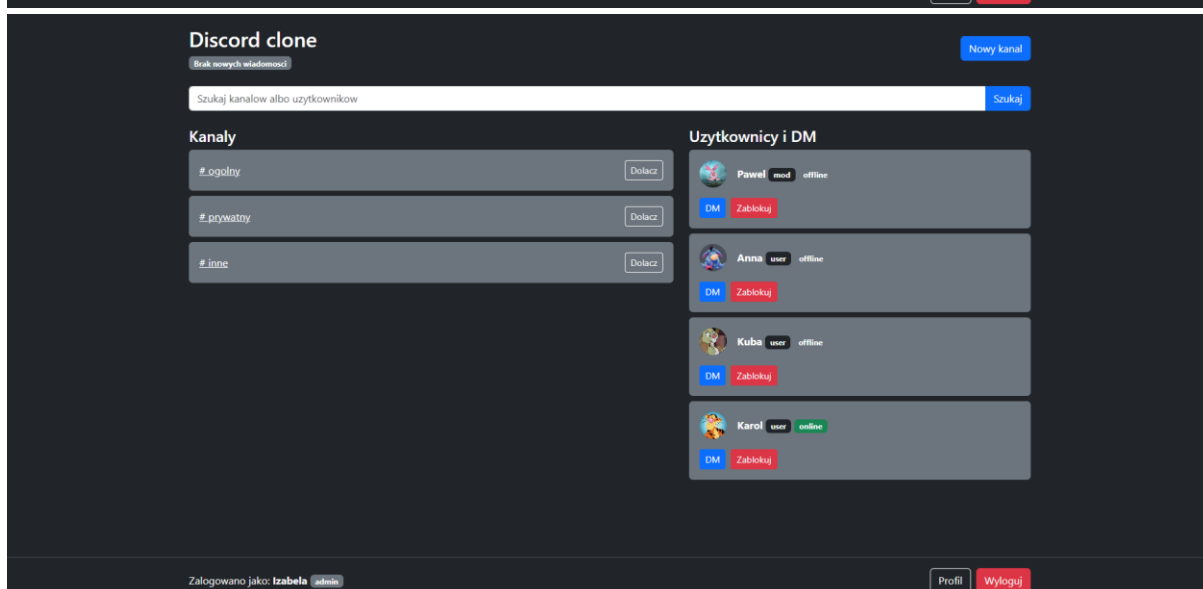
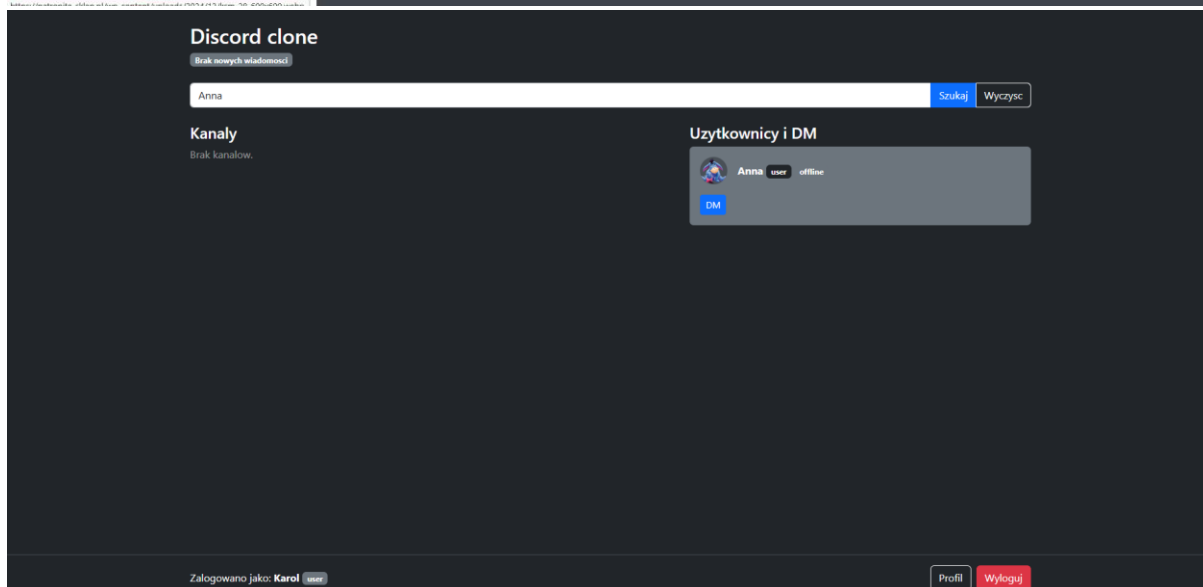
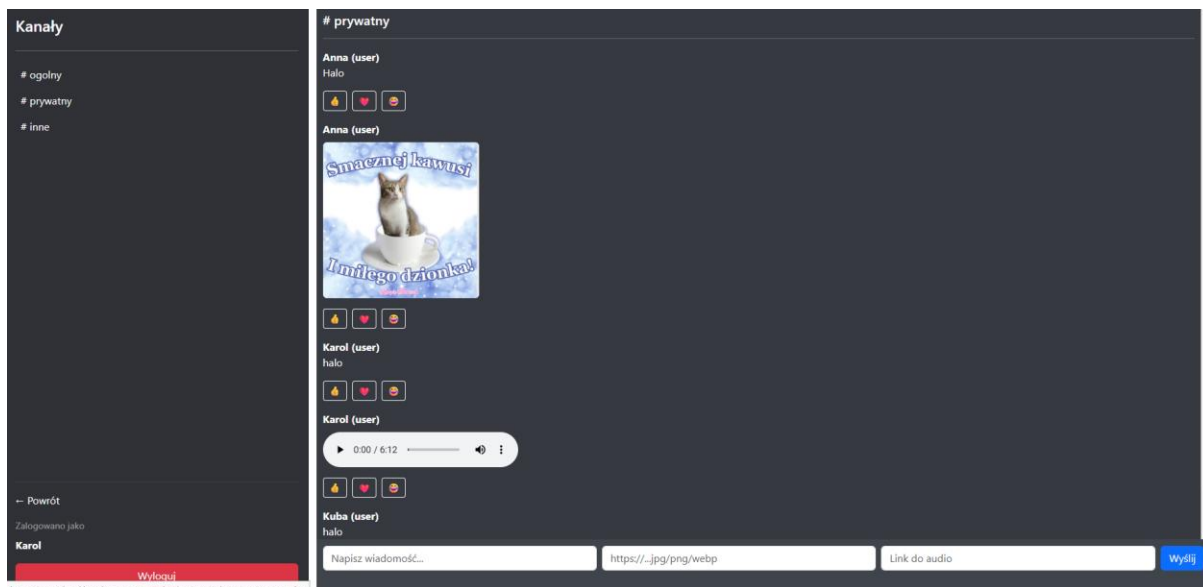
```
139  # Static files (CSS, JavaScript, Images)
140  # https://docs.djangoproject.com/en/5.2/howto/static-files/
141
142  STATIC_URL = 'static/'
143  STATIC_ROOT = BASE_DIR / 'staticfiles'
144  MEDIA_URL = '/media/'
145  MEDIA_ROOT = Path(os.environ.get('MEDIA_ROOT', BASE_DIR / 'media'))
146
147  # Default primary key field type
148  # https://docs.djangoproject.com/en/5.2/ref/settings/#default-auto-field
149
150  DEFAULT_AUTO_FIELD = 'django.db.models.BigAutoField'
151  LOGIN_URL = '/login/'
152  LOGIN_REDIRECT_URL = '/'
153  LOGOUT_REDIRECT_URL = '/login/'
154  AUTH_USER_MODEL = 'chat.User'
```

views.py

```
117 @login_required
118 def join_channel(request, id):
119     channel = Channel.objects.get(id=id)
120     channel.members.add(request.user)
121     return redirect(f'/channel/{id}/')
122
123 @login_required
124 def channel(request, id):
125     mark_online(request.user)
126     channel = Channel.objects.get(id=id)
127     channels = Channel.objects.all()
128     messages = add_reaction_counts(list(Message.objects.filter(channel=channel)))
129
130     if request.method == "POST":
131         if request.user.is_blocked:
132             return HttpResponse("Twoje konto jest zablokowane.")
133
134         content = request.POST.get('content', '').strip()
135         image_url = request.POST.get('image_url', '').strip()
136         audio_url = request.POST.get('audio_url', '').strip()
137
138         if content or image_url or audio_url:
139             Message.objects.create(
140                 user=request.user,
141                 channel=channel,
142                 content=content,
143                 image_url=image_url,
144                 audio_url=audio_url
145             )
146             return redirect(f'/channel/{id}/')
147
148     return render(request, 'channel.html', {
149         'channel': channel,
150         'channels': channels,
151         'messages': messages
152     })
153
```

7. Przykłady działania programu





8. Podsumowanie

Celem projektu było stworzenie webowej aplikacji komunikacyjnej inspirowanej funkcjonalnością platformy Discord. Projekt został zrealizowany z wykorzystaniem frameworka Django oraz technologii webowych umożliwiających budowę nowoczesnej aplikacji internetowej.

W ramach projektu zaimplementowano najważniejsze funkcjonalności komunikatora internetowego, takie jak system rejestracji i logowania użytkowników, obsługa ról i uprawnień, komunikacja w kanałach publicznych oraz wiadomości prywatne. Aplikacja umożliwia również reagowanie na wiadomości, zarządzanie użytkownikami oraz podstawową moderację treści. Projekt został przygotowany zgodnie z architekturą MVT oraz dobrymi praktykami tworzenia aplikacji webowych. Zastosowano mechanizmy walidacji danych, obsługę autoryzacji użytkowników oraz responsywny interfejs dostosowany do urządzeń mobilnych i komputerowych.

Realizacja projektu pozwoliła na praktyczne wykorzystanie wiedzy związanej z programowaniem aplikacji internetowych, obsługą baz danych, systemem uprawnień oraz projektowaniem interaktywnych interfejsów użytkownika. Aplikacja może stanowić podstawę do dalszego rozwoju i rozszerzenia o dodatkowe funkcjonalności, takie jak komunikacja w czasie rzeczywistym z wykorzystaniem WebSocketów, powiadomienia czy obsługa kanałów głosowych.